Tab 1

Got it — no more fix-chasing. Here's the full feature set, area by area, based on what's actually implemented in the code.

Browser

- **Tabbed browsing** — multiple tabs, each with its own persistent WebView (not destroyed on switch), tab switcher grid with live thumbnails, session-restored across app restarts (`BrowserSessionRepository`)
- Address bar with URL/search-term detection, desktop-site toggle (separate UA), bookmarks (add/remove/list), browsing history (auto-recorded, searchable list, per-entry delete, clear all)
- Speed dial / shortcuts grid on the new-tab page, including custom user-added shortcuts with favicon caching
- Favicon fetching per tab and per shortcut
- Long-press-on-link menu: open in new tab, copy link, share link, add to speed dial
- New-tab popup confirmation dialog (open/block) for any page-triggered `target="_blank"`/`window.open()`
- Native `<video>` fullscreen support for a page's own player (not Viowser's dedicated one)
- File upload support (`<input type=file>`) and file downloads via the system download flow
- Page load error screen with retry, instead of WebView's bare error page
- Pause-on-background: all tab media pauses when the app backgrounds, the tab switcher opens, or you leave the browser route
- Dark/light top-bar icon handling per screen, edge-to-edge layout

Video capture ("the grabber")

- Passive **network-level sniffing** of every request the WebView makes (native `shouldInterceptRequest`) to catch HLS/DASH manifests, direct media files, and audio
- **In-page JS sniffing** (`video_sniffer.js`) for `<video>`/`<source>` elements and `HTMLMediaElement.play()` calls, including dynamically-added ones via a MutationObserver
- Distinguishes a real capturable stream from "segments flowing but nothing capturable" (MSE/DRM case) and tells the user which situation they're in
- Floating "open in player" / PiP pills that track the currently playing-with-sound video's on-screen position live as the page scrolls/resizes
- Metadata enrichment per detected video: file size, duration, resolution, segment count (`VideoMetadataProbe`)
- Detected-videos bottom sheet listing everything found on the current page

Downloads

- **HLS and DASH adaptive-stream downloading** (no FFmpeg) — manifest parsing, concurrent segment fetching with configurable connection count, retry policy, and speed limiting
- Video+audio muxing into a single playable MP4 via `MediaMuxer` (no re-encoding); solo-track downloads are also remuxed so they end up seekable, not just a raw segment concatenation
- **Direct file downloads** with multi-connection byte-range splitting and resume
- Pause/resume/cancel per download, backed by durable on-disk checkpoints that survive app restart
- Persistent notification + **foreground service** so downloads survive backgrounding and the app being swiped from Recents
- Download destination options: default public Downloads folder, MediaStore (scoped storage), or a user-picked SAF folder
- Settings: max connections, retry count, Wi-Fi-only, speed limit (KB/s)
- Downloads tab merges MediaStore-indexed files with Viowser's own completed-download records, deduplicated

Player ("Hub")

- Dedicated video player screen (Media3/ExoPlayer under the hood) - Home/Library/Music/Downloads tabs
- Gesture controls: left-side vertical drag = brightness (persists across videos/restarts, scoped to only the video layer, not the Hub), middle = seek scrub, right-side vertical drag = real device media volume
- Double-tap seek, long-press for speed control, aspect ratio (tap-cycles Fit/Stretch/Zoom, hold for a picker) — persists across videos/restarts
- Audio/subtitle track selection
- Orientation lock, immersive mode (system bars hide with controls while watching)
- Picture-in-Picture, including a dedicated "open with PiP" entry point from the browser
- Screen wake lock while a video is actually playing
- Stream history ("Home" tab) — replays a past capture by re-sniffing the source page live and auto-advancing into the player once found, rather than trusting a possibly-expired stored media URL
- Direct-URL playback (paste a raw video link and play it) and local-file playback (device-stored videos, independent of anything Viowser downloaded)
- Running-downloads panel inside the player (progress, pause/cancel) so you don't have to leave to check on one

Ad blocking & privacy

- Domain-based blocking (EasyList/EasyPrivacy-scale filter lists + a curated always-on fallback list + user-added custom lists), with per-source rule tracking so removing one list doesn't touch another's contributions
- Background periodic refresh of subscribed filter lists (`HostsRefreshWorker`)
- Cosmetic filtering — hides ad/tracker DOM elements via injected CSS, with a sweep pass for stuff added after the fact

- Service-worker-level interception, so ads served through a page's own service worker (invisible to normal request interception) are still caught
- Blocked requests get a valid-but-inert stub of the right type (empty script/image/JSON/etc.) instead of a bare failure, so a blocked resource doesn't take the surrounding page down with it
- `market://` / `intent://` popunder-redirect leak blocking (the classic "force-open the Play Store" ad trick)
- Settings toggle to turn ad-blocking off entirely, plus a hosts/filter-list management screen (view rule counts, add/remove sources)
- Cookie handling tuned for auth persistence: third-party cookies accepted, explicit flush on backgrounding so a solved challenge/login survives a killed process

Settings & app-level

- Ad-blocker & hosts management screen
- Download settings screen (connections/retries/Wi-Fi-only/speed limit/download folder)
- Player acceleration info (hardware decoding)
- About section (version name/build number), also shown in the browser's menu footer
- Versioned builds going forward (bumped every change)

What's explicitly out of scope (by deliberate decision, not missing)

- No YouTube/Netflix extraction — dropped for good, not just deferred
- No real libmpv — "MPV" branding is Media3/ExoPlayer under the hood
- Pull-to-refresh — removed outright after repeated attempts never reliably worked
- Dark theme toggle exists in the menu but is currently a no-op
- Find-in-page not implemented

That's the full surface as it stands in the code right now.

Tab 2

Given everything I've actually hit debugging this app this session, here's what I'd pick — and why, grounded in the specific pain points, not generic advice.

The one change that would have prevented half of today's bugs: the browser engine

`android.webkit.WebView` is the wrong foundation for a *browser-shaped* product. It's Chromium, but wrapped in an API that hands you almost no real control:

- No native content-blocking hooks — every ad-block mechanism this session (domain matching, cosmetic CSS injection, service-worker interception) had to be hand-rolled and JS-injected, and every one of them has had a real collision bug (the Google Fonts/github.io domain-parser bug, the cosmetic-selector-hides-the-player bug, the duplicate unguarded block in `video_sniffer.js`).
- No control over touch dispatch — the entire long-press/pull-to-refresh/scroll-misfire saga this session is downstream of WebView's Chromium touch handling and Compose's `AndroidView` interop just not cooperating cleanly.
- No TLS/fingerprint control — the 503/Cloudflare-class blocking is a real ceiling with plain WebView, not something app-level header tweaks fix.

I'd use **GeckoView** (Mozilla's embeddable engine — what Firefox for Android runs on) instead. It has an actual native content-blocking API (`ContentBlocking.Settings` — categories, not regex-matched domain lists you maintain by hand), a real WebExtension API (you could genuinely ship uBlock Origin instead of reinventing it), and a different engine fingerprint than stock WebView, which is at minimum a different starting point against site-side bot detection than "obviously a WebView."

Player & downloads: keep Media3, stop reinventing what it already does

ExoPlayer/Media3 is the right call and I'd keep it exactly as-is — it's the best media stack on Android, actively maintained, and this app already uses it well.

What I'd change: Media3 ships its **own offline download APIs** (`DownloadManager`, `HlsDownloader`, `DashDownloader`) that already do manifest parsing, segment fetching, and muxing, battle-tested against real-world streams. This session's custom `HlsStreamDownloader`/`DashStreamDownloader`/`MediaTrackMuxer`/`ConcurrentSegmentWriter` is a lot of hand-built, harder-to-verify surface area solving a problem Google already solved. I'd build on Media3's own download stack and spend that engineering budget elsewhere.

UI: Compose + Material 3 is right, but designed in from day one, not retrofitted

Keep Jetpack Compose and Material 3 — for a genuinely beautiful, modern Android UI this is the correct toolkit, and this app already uses it reasonably well (the motion system, press-scale interactions). What I'd do differently building fresh:

- **Edge-to-edge and WindowInsets handled from the first screen**, not bolted on later — today's status-bar-overlap bugs exist because insets got added screen-by-screen after the fact instead of being part of the base layout scaffolding from day one.
- **Predictive back gesture** (Android 14+) designed in from the start instead of the `BackHandler`-per-screen juggling that caused several of this session's nav bugs (the "returns to wrong screen" issues came from ad-hoc back-stack logic, not a systematic navigation model).
- **Dark theme and accessibility from day one** — dark theme is a known no-op here today, and it's much cheaper to design tokens for both themes upfront than retrofit hardcoded hex colors across every screen later (which is exactly this codebase's current state).
- Lean into Material 3's newer expressive motion/shape system for the "beautiful" part specifically — richer, more physical transitions than the plain fades this app currently uses between screens.

Architecture

- **Multi-module Gradle** (`:browser`, `:player`, `:download`, `:adblock` as separate modules with real boundaries), not one module where everything can reach everything. This is the direct fix for the meta-problem from earlier in this session — `WebBrowserViewModel.kt` growing to touch tabs, downloads, player-home data, and navigation all at once is exactly how an edit to one feature quietly breaks another.
- **Room** instead of hand-rolled SharedPreferences+JSON repositories (history, bookmarks, sessions) — more robust, queryable, less custom serialization code to have bugs in.
- **Hilt** for DI instead of manual singletons (`StreamDownloadManager.getInstance()`) — more testable, and removes a whole class of "is this actually the same instance everywhere" bugs.
- MVVM + StateFlow, which this app already does — keep it, it's the right pattern here.

The thing I'd prioritize most, honestly

Real instrumented UI tests and an emulator in CI, from the start. Nearly every unresolved bug this session (pull-to-refresh, scroll-misfires, the thumbnail issue) came down to me reasoning about Compose/WebView touch interop from source code with no way to actually run it. That's not a tech-stack pick so much as a process one, but it would have mattered more than any framework choice above.

Platform scope, one open question for you: all of this assumes Android-only stays the goal. If iOS is ever a real target, this calculus changes a lot (GeckoView doesn't exist there,

WKWebView has its own restrictions, App Store policy is stricter about ad-blocking) — worth telling me now if that's on the table, since it'd change the browser-engine recommendation specifically.

Tab 3

The right frame first

This is fundamentally three different apps stitched together: **a browser with content-interception** (find and grab streams), **a media/download engine** (fetch and store adaptive streams), and **a media player/library** (beautiful, polished playback UX). Each of those three has a different "best tool," and the stack should reflect that rather than force one framework to be good at all three.

Platform strategy: native UI and system integration, shared logic underneath

For an app this deep into browser engines, background services, foreground notifications, PiP, and native media pipelines, a cross-platform UI framework (Flutter, React Native) buys you the least where you need the most — those categories all end up needing platform-specific plugins anyway, so the "write once" promise mostly evaporates for exactly this app's core features.

The right split: **Kotlin Multiplatform (KMP)** for the pure logic that doesn't touch platform UI or OS-specific APIs — URL/media classification, ad-block domain matching, download resume/checkpoint state machines, data models, repository interfaces — shared between Android and iOS. Everything that touches the browser engine, the media pipeline, or system integration stays fully native per platform. UI is native per platform too: **Jetpack Compose + Material 3** on Android, **SwiftUI** on iOS. Platform-idiomatic UI consistently reads as more polished than a shared UI layer trying to look native on both — worth the cost of writing it twice if "beautiful" is a real bar.

Browser engine

- **Android: GeckoView** (Mozilla's embeddable engine, what Firefox for Android runs on). It exposes a real, declarative **content-blocking API** and genuine **WebExtension support** — meaning ad-blocking and stream-detection can be built as actual browser extensions using the `webRequest`/`declarativeNetRequest` APIs real ad-blockers use, instead of a hand-rolled native request interceptor plus injected JavaScript fighting the page's own scripts.
- **iOS: WKWebView** is effectively mandatory — Apple requires WebKit for any in-app web rendering, no exceptions. For content blocking, use Apple's sanctioned **Content Blocker extension** format (a declarative JSON ruleset enforced by WebKit itself, not JS injection) — it's literally the mechanism every real iOS ad-blocker uses, and it's more efficient and far more reliable than DOM-level cosmetic hiding.

Media playback and downloads

- **Android: Media3/ExoPlayer**, using its own built-in offline **DownloadManager** (`HlsDownloader`/`DashDownloader`) rather than a custom manifest parser and

muxer — it's Google-maintained, handles adaptive streaming, DRM, PiP, and background playback out of the box.
- **iOS: AVFoundation/AVPlayer**, with **AVAssetDownloadTask** for offline HLS downloads (Apple originated HLS — this is the native, first-class path, including FairPlay DRM support if ever needed).

Data layer

SQLDelight (KMP-shared, typed SQL, compiles to native drivers on both platforms) for history, bookmarks, library, and download state — a real queryable database, not flat JSON blobs in preferences. **Ktor client** (KMP) for shared networking logic where it isn't platform-specific.

Architecture

MVVM/MVI with unidirectional data flow (`StateFlow`/`Combine`), repository pattern over the shared data layer, dependency injection (**Koin**, since it works cleanly across KMP — Hilt is Android-only), and genuine feature-module boundaries (browser, player, downloads, ad-block as separate modules with explicit interfaces between them) so a change in one can't silently reach into another's internals.

Why this combination specifically

It puts native, first-class platform APIs under every feature that's actually hard — content blocking, adaptive-stream downloading, background media — while still sharing the logic that's genuinely platform-independent, and keeps the UI layer fully native for the best achievable polish on each platform. Nothing here is a workaround; every piece is the sanctioned, maintained tool for its job on its platform.

Tab 4

Build order, and why

The riskiest, least-proven part of this stack is the browser-engine + content-blocking integration (GeckoView's WebExtension support, WKWebView's Content Blocker limits) — so that gets built and de-risked first, as a thin vertical slice, before investing in anything else. Everything downstream (player, downloads, library) is comparatively well-trodden ground. Building "polish" or "the full data layer" before the risky part is proven would mean redoing work if the browser engine choice doesn't pan out.

I'd also build Android first, fully, before touching iOS — proving the KMP module boundaries against one real consumer before asking them to serve two.

Phase 0 — Foundations

1. Repo + module skeleton: `:shared` (KMP), `:android-app`, later `:ios-app`. Empty modules, just the dependency graph wired, CI running a build on every push.
2. Pick the KMP source-set layout (`commonMain`/`androidMain`/`iosMain`) and prove it compiles end to end with one trivial shared function called from the Android app. Nothing real yet — this is just making sure the toolchain works before anything depends on it.
3. Set up an Android emulator (and later iOS simulator) in CI immediately. Given how much of this class of app can only be verified by actually running it (touch handling, gesture interop, real page loads), this is infrastructure, not an afterthought at the end.

Phase 1 — Browser shell + content blocking (the risky part, first)

4. Embed GeckoView in a bare Android screen: load a URL, show it, nothing else. Confirm it actually renders real sites correctly on a real device/emulator.
5. Build one WebExtension for stream detection using `webRequest` — the extension observes network responses and reports back manifest/media URLs to the host app. Get this working against 3–4 real streaming sites before building anything else on top of it. This is the single highest-uncertainty piece of the whole app; proving it early means the rest of the architecture can assume it works.
6. Build a second WebExtension (or extend the first) for content blocking using `declarativeNetRequest` plus a maintained filter-list format (parse an existing EasyList-compatible list rather than hand-rolling one). Verify against real ad-heavy sites, checking specifically that legitimate site functionality survives — this is where most ad-blocker bugs live, so test it adversarially now, not after it's buried under other features.
7. Only now build real browser chrome around it: tabs, address bar, history, bookmarks — standard UI work with low technical risk, safe to build once the engine underneath is trusted.

Phase 2 — Shared logic and data layer

8. Move the models that came out of Phase 1 (detected media, request classification, filter-list rules) into `:shared` as KMP common code, with Android as the only real consumer for now.
9. Stand up SQLDelight schemas for history, bookmarks, library entries, and download checkpoints. Wire repositories in `:shared` over that schema.
10. Wire Koin for DI across `:shared` and `:android-app`, replacing any ad-hoc singletons from the throwaway Phase 1 code.

Phase 3 — Player

11. Integrate Media3/ExoPlayer as a standalone screen that can play a raw URL, independent of the browser — prove playback, adaptive switching, and background/PiP behavior in isolation before wiring it to anything the browser detects.
12. Connect Phase 1's detected-media events to this player — tapping a detected stream in the browser hands off to it. This is the first real cross-feature integration point, and worth its own testing pass specifically for the browser↔player handoff (this is exactly the class of interaction that tends to grow subtle nav/state bugs if rushed).

Phase 4 — Downloads

13. Integrate Media3's `HlsDownloader`/`DashDownloader` against the SQLDelight-backed checkpoint schema from Phase 2. Build pause/resume/cancel against real interrupted downloads (kill the process mid-download, confirm resume) before building any UI for it.
14. Foreground service + persistent notification for in-progress downloads, wired to the same download-manager instance the UI reads from — get the single-source-of-truth question right structurally now, since it's a common source of state-drift bugs later.
15. Download destination handling (scoped storage / SAF folder picker) — deliberately last within this phase, since it's UI/permissions plumbing around an engine that's already proven.

Phase 5 — Library / Hub

16. Build the Hub as its own top-level surface reading from the Phase 2 repositories (history, downloaded files, library) — by this point every data source it needs already exists and is tested, so this phase is close to pure UI work.
17. Wire the "replay from history" flow (re-open a past capture, re-detect, auto-resume into the player) — the second cross-feature integration point, given the same care as step 12.

Phase 6 — Polish pass

18. Dark theme and Material 3 Expressive motion across every screen built so far — deliberately done as one dedicated pass across the whole app rather than screen-by-screen as you go, so it's consistent instead of half-migrated.
19. Edge-to-edge/inset handling audited across every screen as a single pass, same reasoning.
20. Predictive back gesture wired at the navigation-graph level, not per-screen `BackHandler`s bolted on individually.
21. Accessibility pass (content descriptions, TalkBack, dynamic type) across the whole app.

Phase 7 — Hardening

22. Instrumented UI tests specifically for the interaction-heavy paths: browser gesture handling, player controls, download pause/resume under real interruption. This is the class of bug that's cheapest to catch with an actual test now and most expensive to diagnose blind later.
23. A real device/OS-version matrix in CI (not just one emulator image) — WebEngine and media behavior both vary meaningfully across OEM skins and OS versions.

Phase 8 — iOS

24. Only now start the iOS app: wire `:shared` as a real second consumer, build the WKWebView + Content Blocker equivalents of Phase 1, AVFoundation/AVAssetDownloadTask equivalents of Phases 3–4, and SwiftUI screens mirroring Phase 5–6. Every architectural boundary from Android either holds up here or reveals itself as Android-specific and needs adjusting — and it's far cheaper to find that out now, with one real feature set to validate against, than if iOS had been built in parallel from day one against a moving target.

This ordering front-loads everything uncertain and defers everything well-understood, which is the opposite of building UI polish first — polish is cheap to redo, a wrong browser-engine bet is not.